



南開大學
Nankai University

计算机学院
计算机体系结构实验报告

五级流水复习和分析

姓名：王一诺

学号：2312331

专业：计算机科学与技术

指导老师：董前琨

2025 年 10 月 13 日

目录

1 实验目的	2
2 实验要求	2
3 实验原理图	2
4 现有流水线问题分析	4
4.1 取消 Primitives Output Register	4
4.2 修改 IF_over 逻辑	5
4.3 总结修改逻辑	6
5 新增加指令分析	7
5.1 MULT（乘法指令）	7
5.2 MFHI（从 HI 读取）	13
5.3 MFLO（从 LO 读取）	15
5.4 MTHI（向 HI 写入）	16
5.5 MTLO（向 LO 写入）	18
5.6 MFC0（从 CP0 读取）	18
5.7 MTC0（向 CP0 写入）	20
5.8 SYSCALL（系统调用）	23
5.9 ERET（异常返回）	26
5.10 新增指令总结	28
6 总结感想	28

1 实验目的

本次实验旨在通过对一个已有的五级流水线 CPU 源码进行 bug 修改、分析和调试，实现以下目标：

1. 深入理解五级流水线（IF、ID、EX、MEM、WB）的工作原理、数据通路和控制逻辑。
2. 熟悉并掌握流水线 CPU 的原理和设计。
3. 通过修复源码中的 Bug，提升 Verilog 代码调试能力和对 CPU 内部复杂控制信号交互的理解。
4. 参考 MIPS 指令集，对新增指令进行具体分析，包括运算、乘除、转移和访存四类指令，从而全面掌握指令在流水线各阶段的实现细节。
5. 培养对 CPU 设计的兴趣，加深对 CPU 现有架构的理解和深思。

2 实验要求

请参考实验指导手册，完成五级流水线 CPU 工程的顺利运行，完成如下工作：

- 1、修改原始代码 source code 中的 bug，保证指令运行正确
- 2、针对五级流水线中新增加的指令，逐级分析指令执行过程和执行结果，梳理流水线知识点
- 3、将上述两部分内容写入实验报告提交。

3 实验原理图

本实验中实现的五级流水线 CPU，其核心数据通路和控制单元如下图所示。CPU 将指令执行过程划分为取指（FETCH）、译码（DECODE）、执行（EXE）、访存（MEM）和写回（WB）五个阶段，并通过四级流水线寄存器（IF/ID, ID/EXE, EXE/MEM, MEM/WB）在阶段间传递数据和控制信号。

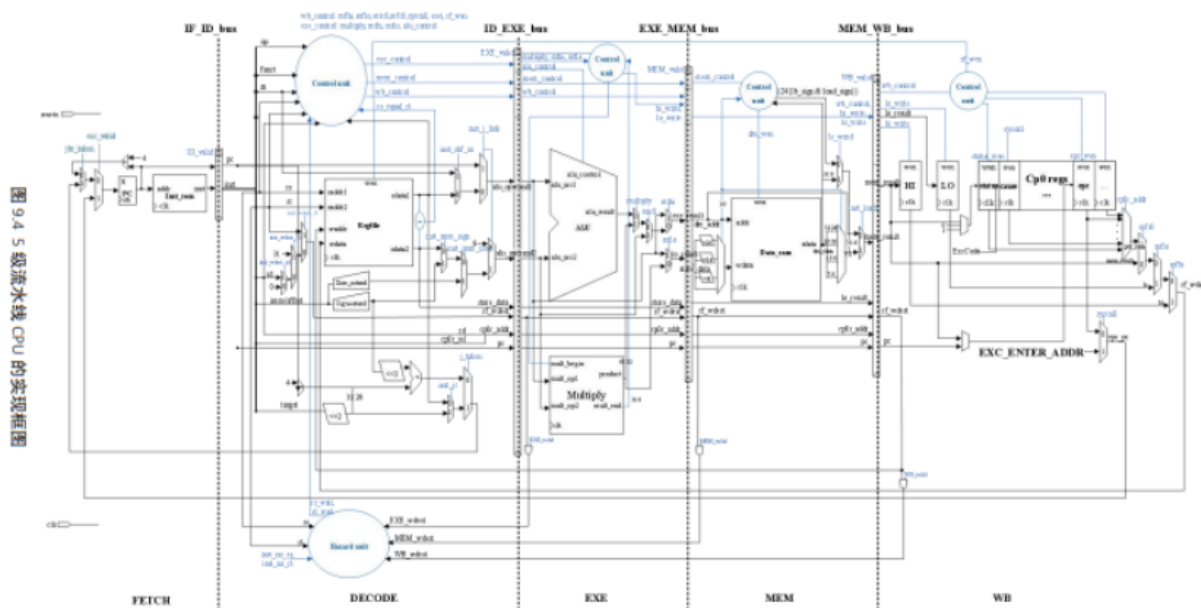


图 3.1: 五级流水线 CPU 详细框图

一个更简化的静态模型如下，它清晰地展示了各阶段与主要部件（指令 ROM、寄存器堆、数据 RAM）的交互关系。

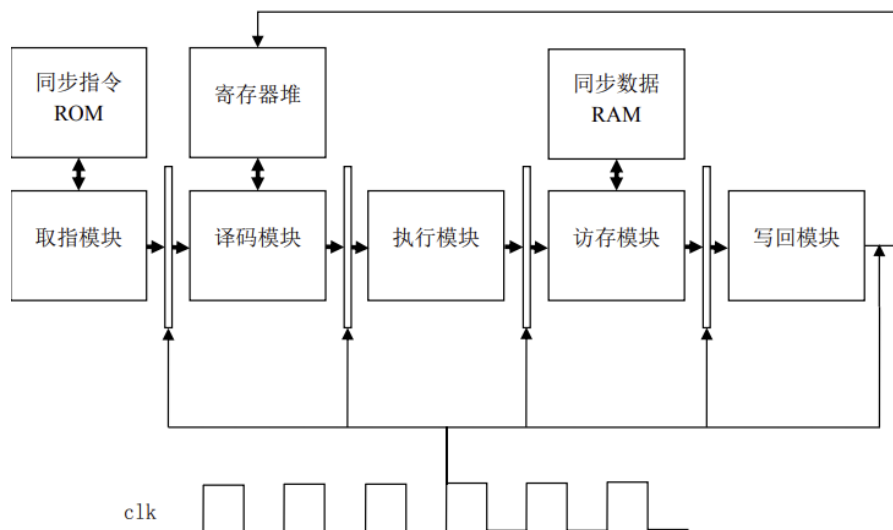


图 9.2 静态 5 级流水 CPU 的大致框图

图 3.2: 静态五级流水 CPU 功能框图

4 现有流水线问题分析

尝试对 source_code 进行仿真，波形图如下：

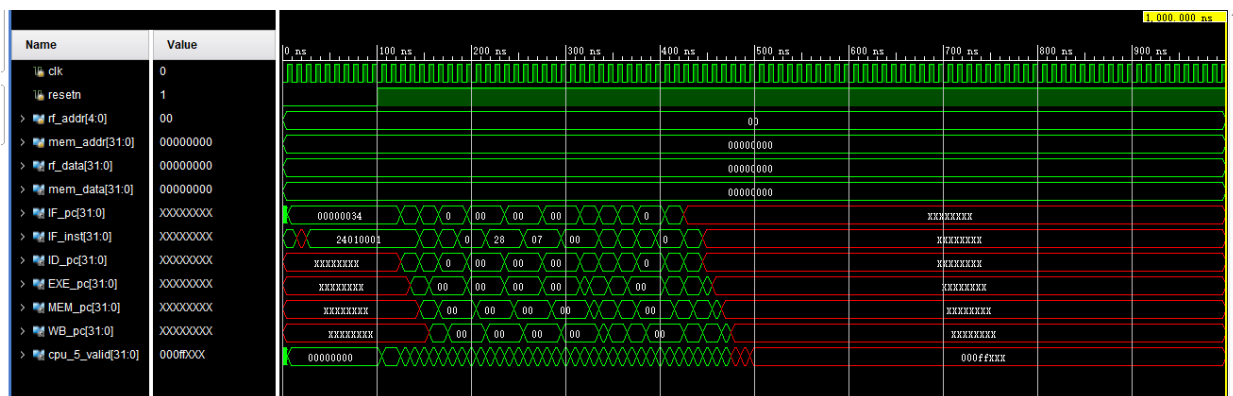


图 4.3: 源代码错误仿真波形

经过分析，发现是指令 ROM 的读取延迟与 IF_over 信号生成时机的不匹配导致的。有两种方案解决该问题：

4.1 取消 Primitives Output Register

在配置 IP 核时，取消勾选 Primitives Output Register 的选项，如下图所示：

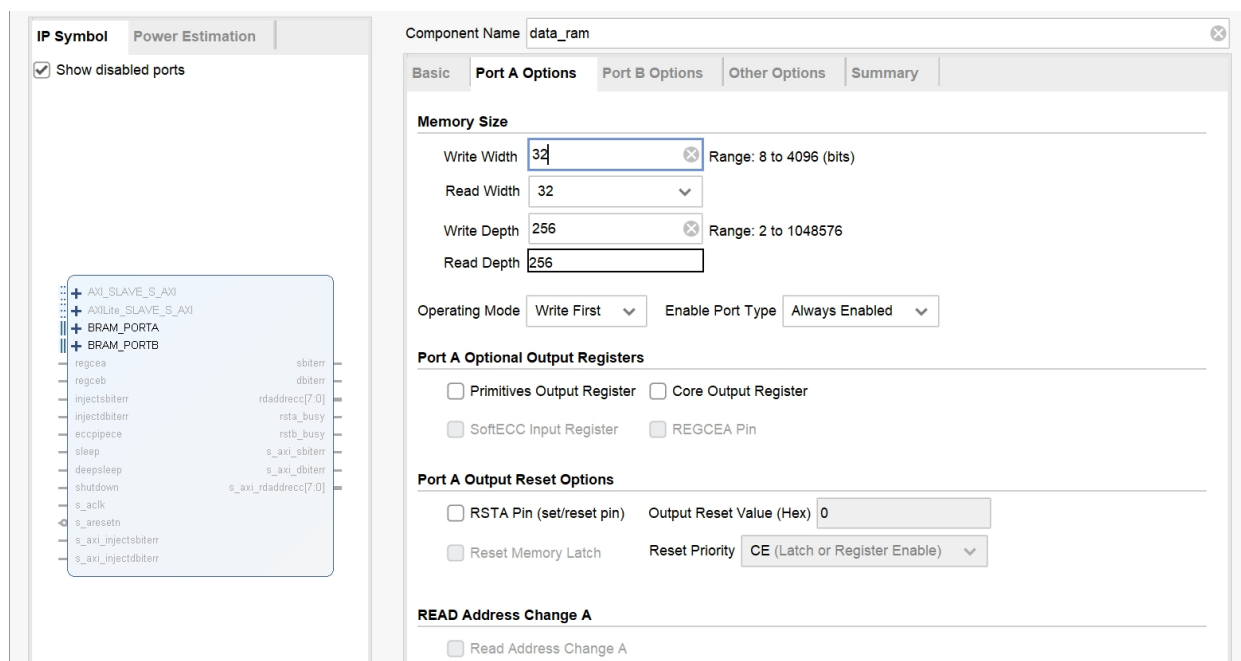


图 4.4: 修改 data_ram

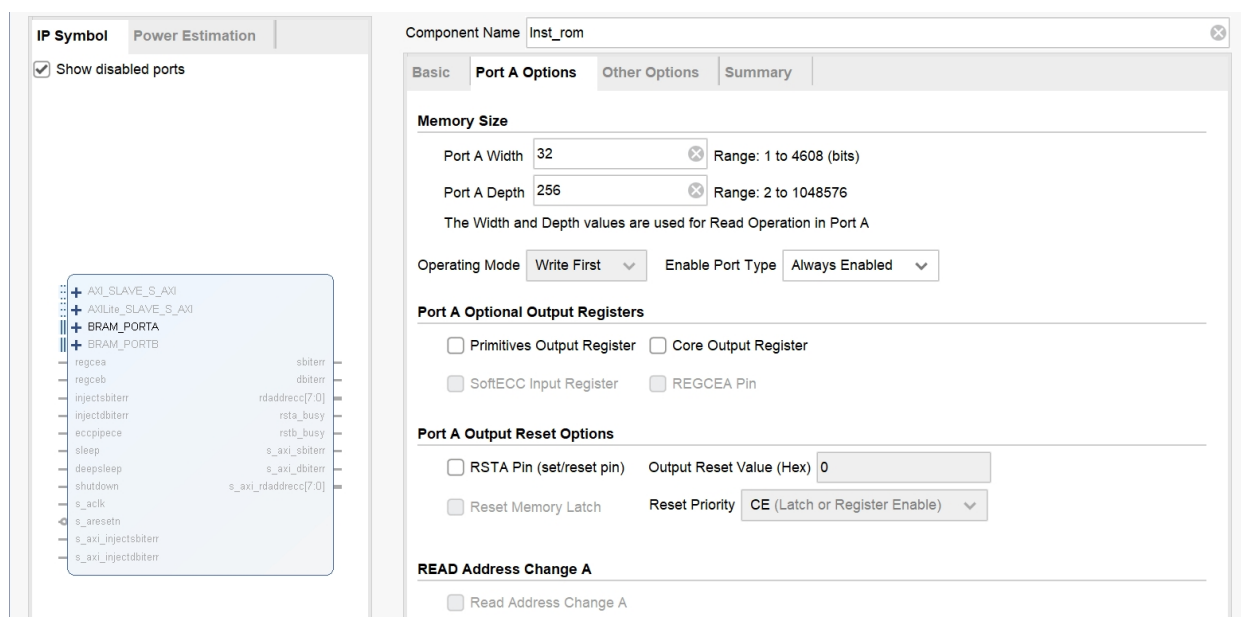


图 4.5: 修改 inst_rom

勾选此选项时：IP 核在 BRAM 原语输出端额外增加一级寄存器；

取消勾选后：只有 BRAM 内部的地址寄存器。使 ROM 变成真正的单周期延迟，保证“取数据时，有一拍延时”，让 IF_over 信号与指令数据真正可用的时刻同步。

4.2 修改 IF_over 逻辑

找到 fetch.v 中的 IF 执行逻辑：

Listing 1: fetch.v-old

```

1  //-----{IF执行完成}begin
2  //由于指令rom为同步读写的，
3  //取数据时，有一拍延时
4  //即发地址的下一拍时钟才能得到对应的指令
5  //故取指模块需要两拍时间
6  //故每次PC刷新，IF_over都要置0
7  //然后将IF_valid锁存一拍即是IF_over信号
8  always @(posedge clk)
9  begin
10     if (!resetn || next_fetch)
11     begin
12         IF_over <= 1'b0;
13     end
14     else
15     begin
16         IF_over <= IF_valid;
17     end
18 end
19 //如果指令rom为异步读的，则IF_valid即是IF_over信号，
20 //即取指一拍完成

```

```
21 //-----{IF 执行完成}end
```

修改为:

Listing 2: fetch.v-new

```
1  reg temp;
2  always @(posedge clk)
3  begin
4      if (!resetn || next_fetch)
5      begin
6          IF_over <= 1'b0;
7          temp<=1'b0;
8      end
9      else if(!temp)
10     begin
11         temp<=1'b1;
12     end
13     else
14     begin
15         IF_over <= IF_valid;
16     end
17 end
```

这样，通过 temp 信号增加 1 拍等待，即通过软件逻辑补偿额外的 1 拍延迟，使 IF_over 信号与数据真正可用时刻对齐。

4.3 总结修改逻辑

两种方案殊途同归：都让 IF_over 信号与指令数据真正可用的时刻同步

1. 方案 1：让硬件适配代码假设（1 拍延迟）
2. 方案 2：让代码适配硬件现实（2 拍延迟）

维度	方案 1（硬件配置）	方案 2（代码逻辑）
修改对象	IP 核配置	Verilog 控制逻辑
ROM 延迟	2 拍 → 1 拍	保持 2 拍
IF_over 时机	保持原逻辑	延后 1 拍
本质	消除硬件多余延迟	适配硬件实际延迟

表 1: 对比总结

修改后:

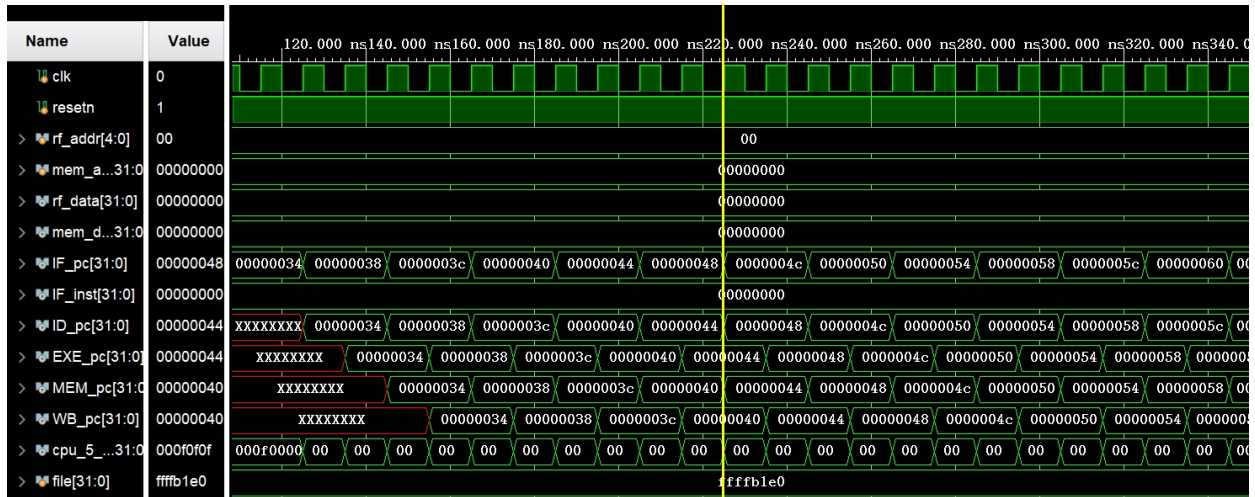


图 4.6: 改后仿真波形

5 新增加指令分析

根据实验指导书附录，该实验新增了 9 条指令，接下来我将结合对应具体代码逐级分析指令执行过程和执行结果。

5.1 MULT（乘法指令）

指令概述与功能 MULT 指令是 MIPS 架构中的有符号乘法指令，它将两个 32 位有符号整数相乘，产生一个 64 位的有符号乘积。乘积的高 32 位存储在 HI 寄存器中，低 32 位存储在 LO 寄存器中。这是一条 R 型指令，指令格式为 000000_sssss_ttttt_00000_00000_011000，其中 rs 和 rt 字段指定两个源操作数寄存器，rd 和 sa 字段必须为 0，功能码 funct 为 011000。

IF 级（取指阶段） 相关代码：

Listing 3: IF

```

1 // fetch.v
2 // 从指令存储器读取MULT指令
3 assign inst_addr = pc; // 将PC送到指令ROM
4 // 下一周期得到指令：000000_sssss_ttttt_00000_00000_011000

```

在取指阶段，程序计数器 PC 保存着当前 MULT 指令的地址。这个地址通过 inst_addr 信号线被送到指令 ROM 模块。由于实验中使用的是同步读写的指令存储器，地址在当前时钟周期的上升沿被锁存到 ROM 中，实际的指令数据要到下一个时钟周期才能读出。在这个等待期间，IF_over 信号保持为低电平，表示取指尚未完成。

PC 的计算遵循特定的优先级规则。首先检查是否有异常发生，如果 exc_valid 信号有效，PC 将跳转到异常入口地址。其次检查是否有分支跳转，如果 jbr_taken 信号有效，PC 将跳转到 jbr_target 指定的目标地址。如果都不满足，PC 就简单地加 4 指向下一条顺序指令。这个 next_pc 的计算是组合逻辑，但实际更新 PC 寄存器要等到 next_fetch 信号有效时，这个信号实际上就是 IF_allow_in 信号。对于 MULT 指令来说，它本身不是跳转指令，所以在正常情况下会按顺序取指。指令存储器在下

一个周期返回 32 位的指令编码，此时 IF_over 信号变为高电平，表示取指完成。IF 模块将 PC 值和指令 inst 打包成 64 位的 IF_ID_bus 信号，准备传递给译码阶段。

ID 级（译码阶段） 相关代码：

Listing 4: ID

```

1 // decode.v
2 // 指令识别
3 assign inst_MULT = op_zero & (rd==5'd0) & sa_zero & (funct == 6'b011000);
4
5 // 设置控制信号
6 assign multiply = inst_MULT; // 乘法标志置1
7
8 // ALU操作数准备
9 assign alu_operand1 = rs_value; // 第一个乘数
10 assign alu_operand2 = rt_value; // 第二个乘数
11
12 // 写回控制
13 assign rf_wen = 1'b0; // MULT不直接写回寄存器
14 assign rf_wdest = 5'd0;
15
16 // 数据相关检测
17 assign rs_wait = (rs!=5'd0) & ((rs==EXE_wdest) | (rs==MEM_wdest) |
18 (rs==WB_wdest));
19
20 assign rt_wait = (rt!=5'd0) & ((rt==EXE_wdest) | (rt==MEM_wdest) |
21 (rt==WB_wdest));
22
23 // ID_EXE总线打包
24 assign ID_EXE_bus = {multiply, // 1'b1
25 mthi, mtlo, // 都为0
26 alu_control, // 12'b0
27 alu_operand1, // rs_value
28 alu_operand2, // rt_value
29 ...};

```

译码阶段接收到 IF_ID_bus_r 锁存的数据，这包含了 PC 值和 32 位指令编码。译码器首先将指令分解为各个字段：操作码 op、源寄存器 rs 和 rt、目标寄存器 rd、移位量 sa、功能码 funct 等。对于 MULT 指令，译码器检测到 op 字段全为 0，rd 字段为 0，sa 字段为 0，且 funct 字段等于 011000，因此 inst_MULT 信号被置为高电平。

MULT 指令的识别逻辑要求 op 为 0（表示 R 型指令），rd 为 0（因为 MULT 不直接写通用寄存器），sa 为 0（MULT 不使用移位量），funct 等于 011000。

一旦识别出 MULT 指令，译码器就开始设置各种控制信号。multiply 信号被置为 1，这是一个关键信号，它会一直传递到执行阶段，告诉乘法器模块开始工作。mthi 和 mtlo 信号都为 0，因为 MULT 本身不是直接向 HI/LO 写入的指令，而是通过乘法运算的结果自动更新这两个寄存器。

ALU 控制信号 alu_control 的 12 位全部为 0，因为 MULT 指令不需要使用 ALU 进行任何算术或逻辑运算。ALU 的作用在 MULT 指令中被完全旁路了。然而，alu_operand1 和 alu_operand2 仍

然需要设置，它们分别被赋值为 `rs_value` 和 `rt_value`，也就是从寄存器堆读出的两个源操作数的值。这两个操作数会被传递到执行阶段，供乘法器使用。

寄存器堆的读取是通过 `rs` 和 `rt` 信号完成的，这两个信号直接来自指令的相应字段。寄存器堆是异步读的，所以在同一个周期内，只要给出地址，就能得到数据。`rs_value` 和 `rt_value` 就是读出的结果。但是这里有一个数据相关性的问题。

数据相关性检测是译码阶段的核心功能之一。如果 `MULT` 指令需要读取的 `rs` 或 `rt` 寄存器正好是前面某条还在流水线中执行的指令要写回的目标寄存器，那么就存在数据相关。具体来说，`rs_wait` 信号会检查 `rs` 是否等于 `EXE_wdest`、`MEM_wdest` 或 `WB_wdest` 中的任何一个。如果相等且 `rs` 不为 0 号寄存器，`rs_wait` 就会置为 1。`rt_wait` 的检测逻辑类似。

当存在数据相关时，`ID_over` 信号会保持为 0，阻止指令向下一级流水。这实际上是一种流水线停顿机制。译码阶段会一直等待，直到前面的指令完成写回，数据相关解除，`EXE_wdest`、`MEM_wdest` 或 `WB_wdest` 中不再包含 `rs` 或 `rt`，此时 `rs_wait` 和 `rt_wait` 都变为 0，`ID_over` 才会变为 1，允许 `MULT` 指令进入执行阶段。

对于 `MULT` 指令，`rf_wen` 被设置为 0，`rf_wdest` 被设置为 0，因为 `MULT` 不写通用寄存器堆。它的结果会写入 `HI` 和 `LO` 这两个特殊寄存器。`mem_control` 信号全为 0，表示不进行内存访问。`store_data` 无效，因为不需要存储。

所有这些控制信号和数据被打包成 167 位的 `ID_EXE_bus`。这个总线包含了执行阶段需要的所有信息：`multiply` 标志、`mthi/mtlo` 标志（都为 0）、`alu_control`（全 0）、两个乘数 `operand1` 和 `operand2`、内存控制信号、写回控制信号、以及 `PC` 值。当 `ID_over` 和 `EXE_allow_in` 都为 1 时，`ID_EXE_bus` 的内容会在时钟上升沿被锁存到 `ID_EXE_bus_r` 寄存器中，供执行阶段使用。

EXE 级（执行阶段） 相关代码：

Listing 5: EXE

```

1 // exe.v
2 // 解包总线
3 assign {multiply, mthi, mtlo, alu_control,
4         alu_operand1, alu_operand2, ...} = ID_EXE_bus_r;
5
6 // 启动乘法器
7 assign mult_begin = multiply & EXE_valid;
8
9 // multiply.v - 乘法器模块
10 // 1. 计算操作数符号和绝对值
11 assign op1_sign = mult_op1[31];
12 assign op2_sign = mult_op2[31];
13 assign op1_absolute = op1_sign ? (~mult_op1+1) : mult_op1;
14 assign op2_absolute = op2_sign ? (~mult_op2+1) : mult_op2;
15
16 // 2. 加载被乘数和乘数（第一个周期）
17 always @(posedge clk) begin
18   if (mult_begin) begin
19     multiplicand <= {32'd0, op1_absolute}; // 低32位
20     multiplier <= op2_absolute;
21     product_temp <= 64'd0;

```

```

22     mult_valid <= 1'b1;
23     end
24     end
25
26     // 3. 迭代计算 (多个周期)
27     always @(posedge clk) begin
28         if (mult_valid) begin
29             multiplicand <= {multiplicand[62:0], 1'b0}; // 左移
30             multiplier <= {1'b0, multiplier[31:1]}; // 右移
31             product_temp <= product_temp + partial_product;
32         end
33     end
34
35     assign partial_product = multiplier[0] ? multiplicand : 64'd0;
36
37     // 4. 乘法结束检测
38     assign mult_end = mult_valid & ~(|multiplier); // 乘数全0时结束
39
40     // 5. 结果符号处理
41     assign product = product_sign ? (~product_temp+1) : product_temp;
42
43     // EXE完成控制
44     assign EXE_over = EXE_valid & (~multiply | mult_end); // 等待乘法完成
45
46     // 准备写HI/LO的数据
47     assign exe_result = multiply ? product[63:32] : alu_result; // HI
48     assign lo_result = product[31:0]; // LO
49     assign hi_write = multiply; // 写HI使能
50     assign lo_write = multiply; // 写LO使能
51
52     // EXE_MEM总线打包
53     assign EXE_MEM_bus = {mem_control, // 4'b0
54         store_data, // 无效
55         exe_result, // 高32位乘积
56         lo_result, // 低32位乘积
57         hi_write, // 1'b1
58         lo_write, // 1'b1
59         mfhi, mflo, // 都为0
60         ...};

```

执行阶段从 ID_EXE_bus_r 中解包出所有控制信号和数据。对于 MULT 指令，关键的信号是 multiply 为 1，alu_operand1 和 alu_operand2 包含了两个要相乘的数。

执行阶段首先生成 mult_begin 信号，这个信号等于 multiply 与 EXE_valid 的逻辑与。只有当执行级有效且 multiply 为 1 时，乘法才会真正开始。mult_begin 信号被送到乘法器模块 multiply_module，触发乘法运算的启动。

乘法器模块的设计采用了迭代累加算法，这是一种低效率但实现简单的算法。首先，乘法器需要处理操作数的符号。两个源操作数 mult_op1 和 mult_op2 可能是负数，乘法器提取它们的符号位 op1_sign 和 op2_sign（即最高位），然后计算绝对值。对于正数，绝对值就是它本身；对于负数，绝

对值是取反加 1 的结果。这样，op1_absolute 和 op2_absolute 就得到了两个操作数的绝对值。

在 mult_begin 信号到来的那个时钟周期，乘法器进行初始化。multiplicand 寄存器被加载为 32'd0，op1_absolute，这是一个 64 位的值，低 32 位是第一个操作数的绝对值，高 32 位全 0。multiplier 寄存器被加载为 op2_absolute，这是一个 32 位的值。product_temp 寄存器被清零，它将用于累加部分积。mult_valid 寄存器被置为 1，表示乘法正在进行。

从下一个时钟周期开始，乘法器进入迭代循环。每个周期，乘法器检查 multiplier 的最低位。如果这一位是 1，说明当前的被乘数需要累加到部分积中，所以 partial_product 等于 multiplicand；如果这一位是 0，partial_product 就是 0。product_temp 在每个周期累加 partial_product。

同时，multiplicand 每周期逻辑左移一位，相当于乘以 2。这对应了乘法的原理：如果乘数的某一位是 1，被乘数要左移相应的位数后累加。multiplier 每周期逻辑右移一位，这样下一个周期就会检查下一位。

这个迭代过程一直持续到 multiplier 的所有位都变成 0 为止。此时 mult_end 信号变为 1，mult_end 的条件是 mult_valid 为 1 且 multiplier 的所有位都是 0（用或归约运算符检测）。当 mult_end 为 1 时，mult_valid 在下一个周期被清零，表示乘法结束。

在整个乘法过程中，product_sign 寄存器记录了乘积的符号，它等于两个操作数符号的异或。如果两个操作数同号，product_sign 为 0，乘积为正；如果异号，product_sign 为 1，乘积为负。

乘法结束后，最终的 product 是根据符号调整的。如果 product_sign 为 1，product 等于 product_temp 取反加 1（即取补码）；如果为 0，product 直接等于 product_temp。这个 64 位的 product 就是最终的乘法结果。

在执行阶段，EXE_over 信号的生成逻辑需要特别注意。对于普通的 ALU 指令，一个周期就能完成，所以 EXE_over 可以直接等于 EXE_valid。但对于 MULT 指令，需要等待多个周期直到乘法完成。因此，EXE_over 的逻辑是：EXE_valid 与（非 multiply 或 mult_end）的逻辑与。这意味着，如果不是乘法指令，立即完成；如果是乘法指令，必须等到 mult_end 为 1 才完成。这个设计确保了流水线能够正确地停顿等待乘法结果。

exe_result 的选择逻辑也很重要。对于 MULT 指令，exe_result 应该选择 product 的高 32 位，即 product[63:32]，这将被写入 HI 寄存器。lo_result 选择 product 的低 32 位，即 product[31:0]，这将被写入 LO 寄存器。hi_write 和 lo_write 信号都被置为 1，表示需要更新 HI 和 LO 寄存器。

所有这些信息被打包进 EXE_MEM_bus。这个 154 位的总线包含：内存控制信号（全 0，不访问内存）、store_data、exe_result（product 高 32 位）、lo_result（product 低 32 位）、hi_write（1）、lo_write（1）、其他控制信号、以及 PC 值。

MEM 级（访存阶段） 相关代码：

Listing 6: MEM

```

1 // mem.v
2 // MULT指令不访问内存，直接传递
3 assign {mem_control, store_data, exe_result, lo_result,
4         hi_write, lo_write, ...} = EXE_MEM_bus_r;
5
6 assign inst_load = 1'b0;
7 assign inst_store = 1'b0;
8 assign dm_wen = 4'b0000; // 不写内存
9
10 // MEM立即完成

```

```

11     assign MEM_over = MEM_valid; // 非load指令一拍完成
12
13     // 传递到WB
14     assign mem_result = exe_result; // 高32位
15     assign MEM_WB_bus = {rf_wen,      // 1'b0
16         rf_wdest,    // 5'd0
17         mem_result,  // 高32位
18         lo_result,   // 低32位
19         hi_write,    // 1'b1
20         lo_write,    // 1'b1
21         ...};

```

访存阶段对于 MULT 指令来说相对简单,因为 MULT 不涉及内存访问。MEM 模块从 EXE_MEM_bus_r 中解包数据,检查 mem_control 信号,发现 inst_load 和 inst_store 都为 0,说明不需要访问数据 RAM。因此, dm_wen 被设置为 4'b0000,不写内存; dm_addr 和 dm_wdata 的值无关紧要,因为它们不会被使用。

MEM_over 信号的生成依赖于是否有 load 操作。对于 load 指令,由于数据 RAM 是同步读的,需要两个周期才能拿到数据,所以 MEM_over 需要等待。但对于 MULT 指令,inst_load 为 0, MEM_over 直接等于 MEM_valid,意味着访存阶段一个周期就完成了。

mem_result 的选择逻辑中,由于 inst_load 为 0, mem_result 直接等于 exe_result,也就是乘积的高 32 位。这个值将传递到写回阶段。

MEM_WB_bus 将所有需要的信息打包传递给写回阶段: rf_wen(0)、rf_wdest(0)、mem_result(乘积高 32 位)、lo_result(乘积低 32 位)、hi_write(1)、lo_write(1)、以及其他控制信号和 PC 值。当 MEM_over 和 WB_allow_in 都为 1 时,这些信息在时钟上升沿被锁存到 MEM_WB_bus_r 中。

WB 级(写回阶段) 相关代码:

Listing 7: WB

```

1 // wb.v
2 // 解包总线
3 assign {wen, wdest, mem_result, lo_result,
4     hi_write, lo_write, ...} = MEM_WB_bus_r;
5
6 // 写HI寄存器
7 always @(posedge clk) begin
8     if (hi_write) begin
9         hi <= mem_result; // product[63:32]
10    end
11 end
12
13 // 写LO寄存器
14 always @(posedge clk) begin
15     if (lo_write) begin
16         lo <= lo_result; // product[31:0]
17    end
18 end
19

```

```

20 // 不写通用寄存器
21 assign rf_wen = wen & WB_over; // wen=0, 所以rf_wen=0

```

写回阶段是 MULT 指令产生结果的地方。WB 模块从 MEM_WB_bus_r 中解包数据，得到 hi_write 为 1、lo_write 为 1、mem_result 包含高 32 位、lo_result 包含低 32 位。

HI 寄存器和 LO 寄存器是两个 32 位的寄存器，用时序逻辑实现。当 hi_write 信号为 1 时，在时钟上升沿，hi 寄存器被更新为 mem_result 的值，也就是乘积的高 32 位。当 lo_write 信号为 1 时，在时钟上升沿，lo 寄存器被更新为 lo_result 的值，也就是乘积的低 32 位。这两个寄存器会保持这些值，直到下一次被 MULT 指令更新，或者被 MTHI/MTLO 指令显式修改。

对于通用寄存器堆的写入，rf_wen 等于 wen 与 WB_over 的逻辑与。由于 wen 为 0（MULT 不写通用寄存器），rf_wen 也是 0，所以不会向寄存器堆写入任何数据。rf_wdest 为 0，rf_wdata 的值无关紧要。

WB_over 信号直接等于 WB_valid，表示写回阶段一个周期完成。当 WB_over 为 1 且下一级（这是最后一级了）允许时，流水线可以继续前进，下一条指令可以进入写回阶段。

总结 两个源操作数从寄存器堆读出，经过 ID 级传递到 EXE 级，在 EXE 级由乘法器模块进行多周期乘法运算，产生 64 位乘积。乘积的高 32 位和低 32 位分别经过 MEM 级（不做任何处理，只是传递），最后在 WB 级分别写入 HI 和 LO 寄存器。整个过程不涉及 ALU，不涉及内存访问，不写通用寄存器。

5.2 MFHI（从 HI 读取）

指令概述与功能 MFHI, "Move From HI", 它的功能是将 HI 寄存器的内容复制到一个通用寄存器中。这是一条 R 型指令，格式为 000000_00000_00000_00000_00000_010000，其中只有 rd 字段有意义，指定目标寄存器。rs、rt 和 sa 字段全为 0，功能码 funct 为 010000。

HI 寄存器通常配合乘法指令使用。MULT 指令将 64 位乘积的高 32 位存入 HI，程序需要使用 MFHI 指令才能将这个值取出到通用寄存器中，进行后续的运算或存储操作。MFHI 不会修改 HI 寄存器的内容，只是读取。

IF 级（取指阶段） 取指过程与 MULT 指令类似。PC 指向 MFHI 指令的地址，这个地址被送到指令 ROM。由于是同步读，实际的指令要在下一个周期才能读出。对于 MFHI 指令，它本身不影响 PC 的计算，所以取指过程很标准。唯一需要注意的是，如果前面有 MULT 指令正在执行，可能会导致流水线停顿，IF 阶段可能需要等待多个周期才能向前推进。

ID 级（译码阶段） 相关代码：

Listing 8: ID

```

1 // decode.v
2 assign inst_MFHI = op_zero & (rs==5'd0) & (rt==5'd0)
3 & sa_zero & (funct == 6'b010000);
4
5 // 分类到rd目标指令
6 assign inst_wdest_rd = ... | inst_MFHI | ...;
7
8 // 控制信号

```



```

9      assign mfhi = inst_MFHI; // 1'b1
10     assign rf_wen = 1'b1;    // 需要写回
11     assign rf_wdest = rd;    // 目标寄存器
12
13     // ID_EXE总线
14     assign ID_EXE_bus = {multiply, // 1'b0
15         mthi, mtlo, // 都为0
16         ...,
17         mfhi, // 1'b1
18         ...,
19         rf_wen, // 1'b1
20         rf_wdest, // rd
21         pc};

```

译码阶段接收到指令后，开始解析各个字段。对于 MFHI，操作码 op 为 0，rs 为 0，rt 为 0，rd 为某个寄存器号（1 到 31 之间），sa 为 0，funct 为 010000。译码器通过组合逻辑识别出这是 MFHI 指令，inst_MFHI 信号被置为 1。

MFHI 被分类为目标寄存器是 rd 的指令，所以 inst_wdest_rd 信号为 1。这意味着 rf_wdest 将被设置为指令中的 rd 字段。同时，rf_wen 被置为 1，因为 MFHI 需要写回通用寄存器堆。

关键的控制信号是 mfhi 被置为 1，mflo 为 0。这个 mfhi 信号会一路传递到写回阶段，在那里控制写回数据的选择，确保选中的是 HI 寄存器的值而不是其他来源的数据。

ALU 相关的信号在 MFHI 中都不重要。alu_control 的 12 位可以全为 0，因为不需要 ALU 运算。alu_operand1 和 alu_operand2 的值也无关紧要，它们不会被使用。内存控制信号 mem_control 全为 0，因为 MFHI 不访问内存。

所有这些控制信号被打包进 ID_EXE_bus。multiply 为 0（不是乘法），mthi 和 mtlo 都为 0（不是向 HI/LO 写入），mfhi 为 1（关键信号），mflo 为 0，rf_wen 为 1，rf_wdest 为 rd。当 ID_over 变为 1 且 EXE_allow_in 为 1 时，这些信息在时钟上升沿被传递到执行阶段。

EXE 级（执行阶段） 执行阶段对于 MFHI 来说几乎是透明的。EXE 模块从 ID_EXE_bus_r 中解包信号，发现 multiply 为 0，所以不启动乘法器。ALU 也不需要工作，因为 alu_control 全为 0。

EXE_over 立即等于 EXE_valid，因为没有任何多周期操作需要等待。MFHI 在执行阶段本质上只是一个信号传递的过程，没有实际的计算发生。EXE_MEM_bus 打包所有信息传递到访存阶段。关键是 mfhi 信号保持为 1，rf_wen 为 1，rf_wdest 为 rd。

MEM 级（访存阶段） 访存阶段对 MFHI 也是透明的。mem_control 全为 0，所以 inst_load 和 inst_store 都为 0，不访问数据 RAM。dm_wen 为 4'b0000，不写内存。由于不是 load 指令，MEM_over 直接等于 MEM_valid，一个周期完成。mem_result 等于 exe_result，虽然这个值在 MFHI 的情况下无关紧要。MEM_WB_bus 继续传递所有信号。mfhi 保持为 1，rf_wen 保持为 1，rf_wdest 保持为 rd。

WB 级（写回阶段） 相关代码：

Listing 9: WB

```

1      // wb.v
2      assign {wen, wdest, mem_result, lo_result,

```

```

3      hi_write, lo_write, mfhi, mflo, ...} = MEM_WB_bus_r;
4
5      // 选择写回数据
6      assign rf_wdata = mfhi ? hi :           // 选择HI寄存器的值
7      mflo ? lo :
8      mfc0 ? cp0r_rdata : mem_result;
9
10     // 写回寄存器堆
11     assign rf_wen = wen & WB_over;         // 1'b1
12     assign rf_wdest = wdest;               // rd
13     // rf_wdata = hi (HI寄存器的当前值)

```

WB 模块解包 MEM_WB_bus_r, 看到 mfhi 为 1, mflo 为 0, mfc0 为 0。

rf_wdata 的选择逻辑是一个多路选择器。首先检查 mfhi, 如果 mfhi 为 1, rf_wdata 选择 hi 寄存器的当前值。如果 mfhi 为 0 但 mflo 为 1, 选择 lo。如果都不是但 mfc0 为 1, 选择 cp0r_rdata。如果都不是, 选择 mem_result。这是一个优先级编码器的结构。

对于 MFHI, mfhi 为 1, 所以 rf_wdata 直接等于 hi。hi 是一个 32 位寄存器, 保存着上一次 MULT 或 MTHI 指令写入的值。这个值现在被选中作为写回数据。

rf_wen 等于 wen 与 WB_over 的逻辑与。wen 在这里是 1 (从 MEM_WB_bus_r 解包得到的), WB_over 也是 1 (写回阶段一个周期完成), 所以 rf_wen 为 1, 表示允许写寄存器堆。

rf_wdest 等于 wdest, 也就是指令中的 rd 字段。regfile 模块看到 wen 为 1 (这里的 wen 就是 rf_wen), waddr 为 rd, wdata 为 hi 的值, 就会在时钟上升沿将 hi 的值写入 GPR[rd]。

整个过程不涉及 HI 寄存器本身的修改。hi_write 为 0, 所以在时钟上升沿, 即使进入了写 HI 寄存器的 always 块, 条件不满足, hi 保持不变。同样, lo_write 为 0, lo 也保持不变。

总结 MFHI 的数据通路非常简洁: 数据从 WB 级的 HI 寄存器直接通过 rf_wdata 信号线连接到寄存器堆的写数据端口。中间的 IF、ID、EXE、MEM 各级只是传递控制信号, 确保在写回阶段能够正确地选择 HI 作为数据源, 并将其写入正确的目标寄存器。

提问: 如果前面有指令要写 HI 寄存器 (如 MULT), 而 MFHI 紧跟其后会发生什么? 这里存在一个读后写 (WAR) 或写后读 (RAW) 的相关性。

在当前的设计中, HI 和 LO 寄存器的相关性检测并不像通用寄存器那样完善。数据相关检测只针对通用寄存器的 wdest 进行。对于 HI/LO 寄存器, 如果 MULT 在 EXE 阶段, 而 MFHI 在 ID 阶段, MFHI 不会被阻塞, 因为数据相关检测不涉及 HI/LO。这可能导致 MFHI 读到旧的 HI 值, 而不是 MULT 将要写入的新值。

5.3 MFLO (从 LO 读取)

指令概述与功能 MFLO, "Move From LO", 功能是将 LO 寄存器的内容复制到一个通用寄存器中。它与 MFHI 非常相似, 唯一的区别是操作的目标寄存器是 LO 而不是 HI。指令格式为 000000_00000_00000_00000_00000_00000, rd 字段指定目标寄存器, 功能码 funct 为 010010。

LO 寄存器存储乘法结果的低 32 位。在执行完 MULT 指令后, 程序通常需要使用 MFHI 获取高 32 位, MFLO 获取低 32 位, 这样才能得到完整的 64 位乘积。对于某些只关心低 32 位结果的应用, 只需要使用 MFLO。

MFLO 的数据通路与 MFHI 几乎完全对称：数据从 WB 级的 LO 寄存器通过 rf_wdata 连接到寄存器堆的写端口。中间各级只负责传递 mflo 控制信号，确保在写回时选择正确的数据源。不再赘述。

5.4 MTHI（向 HI 写入）

指令概述与功能 MTHI, "Move To HI", 功能是将一个通用寄存器的值写入 HI 寄存器。这与 MFHI 正好相反:MFHI 是从 HI 读到通用寄存器,MTHI 是从通用寄存器写到 HI。指令格式为 000000_sssss_00000_00000_00000。rs 字段指定源寄存器, rt、rd 和 sa 必须为 0, funct 为 010001。

MTHI 通常用于初始化 HI 寄存器, 或者在某些算法中显式地设置 HI 的值。例如, 某些定点运算或长整数运算可能需要手动设置 HI 和 LO。

IF 级（取指阶段） 取指过程标准化, PC 指向 MTHI 指令, 地址送到 ROM, 等待一个周期得到指令。IF_over 变为 1, 指令和 PC 通过 IF_ID_bus 传递到译码阶段。

ID 级（译码阶段） 相关代码:

Listing 10: ID

```

1 // decode.v
2 assign inst_MTHI = op_zero & (rt==5'd0) & (rd==5'd0)
3 & sa_zero & (funct == 6'b010001);
4
5 assign mthi = inst_MTHI; // 1'b1
6 assign rf_wen = 1'b0;    // 不写通用寄存器
7 assign rf_wdest = 5'd0;
8
9 // 需要读rs的值
10 assign alu_operand1 = rs_value; // 要写入HI的数据
11
12 // ID_EXE总线
13 assign ID_EXE_bus = {multiply,          // 1'b0
14     mthi,                                // 1'b1
15     ...,
16     pc};

```

译码器识别 MTHI: op 为 0, rt 为 0, rd 为 0, sa 为 0, funct 为 010001。inst_MTHI 信号置为 1。关键的区别是, MTHI 需要读取源寄存器 rs 的值。rs 字段不为 0, 译码器需要从寄存器堆读取值。rs 信号被送到寄存器堆的 raddr1 端口, rs_value 就是读出的数据。

由于需要读 rs, 数据相关检测变得重要。如果 rs 等于 EXE_wdest、MEM_wdest 或 WB_wdest 中的任何一个, 且 rs 不为 0, rs_wait 就会置为 1, 导致 ID_over 保持为 0, 流水线停顿。只有当前面的指令完成写回, rs_wait 变为 0, ID_over 才会变为 1, MTHI 才能进入执行阶段。

inst_no_rt 为真, 因为虽然 rt 字段在指令中存在, 但 MTHI 不需要读 rt 的值。rt_wait 不会因为 MTHI 而置为 1。

mthi 信号置为 1, 这是关键控制信号。mtlo、mfhi、mflo 都为 0。rf_wen 为 0, 因为 MTHI 不写通用寄存器, 它写的是 HI 这个特殊寄存器。rf_wdest 为 0。

alu_operand1 被设置为 rs_value, 这是要写入 HI 的数据。虽然 ALU 不会真正用到这个数据, 但通过 ALU 操作数通路传递数据是一种简洁的设计选择。alu_operand2 和 alu_control 都不重要。

ID_EXE_bus 打包时, 关键信号是 mthi 为 1, alu_operand1 为 rs_value, rf_wen 为 0, rf_wdest 为 0。当数据相关解除且 ID_over 为 1 时, 这些信号传递到执行阶段。

EXE 级（执行阶段） 相关代码:

Listing 11: EXE

```

1 // exe.v
2 assign exe_result = mthi ? alu_operand1 : // rs_value 作为要写入HI的值
3     mtc0 ? alu_operand2 :
4     multiply ? product[63:32] : alu_result;
5
6 assign hi_write = multiply | mthi; // 1'b1 (关键)
7 assign lo_write = multiply | mtlo; // 1'b0
8
9 assign EXE_over = EXE_valid; // 一拍完成
10
11 assign EXE_MEM_bus = {mem_control, // 4'b0
12     store_data,
13     exe_result, // rs_value (要写入HI)
14     lo_result,
15     hi_write, // 1'b1
16     lo_write, // 1'b0
17     mfhi, mflo, // 都为0
18     ...,
19     rf_wen, // 1'b0
20     rf_wdest, // 5'd0
21     pc};

```

exe_result 的选择逻辑中, mthi 的优先级很高。如果 mthi 为 1, exe_result 直接等于 alu_operand1, 也就是 rs_value。这个值将被传递到后续阶段, 最终写入 HI 寄存器。

hi_write 信号的生成逻辑是: hi_write = multiply | mthi。对于 MTHI 指令, multiply 为 0, mthi 为 1, 所以 hi_write 为 1。这个信号表示需要写 HI 寄存器。lo_write 的逻辑是: lo_write = multiply | mtlo。对于 MTHI, multiply 和 mtlo 都为 0, 所以 lo_write 为 0, 不写 LO 寄存器。

EXE_over 立即等于 EXE_valid, 因为 MTHI 不需要多周期操作。执行阶段一个周期完成。

EXE_MEM_bus 打包时, exe_result 包含 rs_value, hi_write 为 1, lo_write 为 0。这些信号传递到访存阶段。

MEM 级（访存阶段） 访存阶段对 MTHI 是透明的。不访问内存, dm_wen 为 0。mem_result 等于 exe_result, 也就是 rs_value, 这个值将被传递到写回阶段。hi_write 和 lo_write 的值保持不变, 继续在 MEM_WB_bus 中传递。MEM_over 立即等于 MEM_valid, 访存阶段一个周期完成。

WB 级（写回阶段） 相关代码:

Listing 12: WB

```

1 // wb.v
2 // 写HI寄存器

```

```

3    always @(posedge clk) begin
4        if (hi_write) begin // hi_write=1
5            hi <= mem_result; // 将rs_value写入HI
6        end
7    end
8
9    // 不写LO (lo_write=0)
10   // 不写通用寄存器 (rf_wen=0)

```

写回阶段是 MTHI 真正修改 HI 寄存器的地方。WB 模块解包 MEM_WB_bus_r, 得到 hi_write 为 1, mem_result 包含要写入的值 (rs_value)。

HI 寄存器的更新逻辑是一个时序 always 块, 当 hi_write 为 1 时, 在时钟上升沿, hi 被更新为 mem_result, 也就是原来的 rs_value。从此以后, HI 寄存器保存着这个新值, 直到下一次被 MULT 或 MTHI 修改。

由于 rf_wen 为 0 (从 MEM_WB_bus_r 解包得到的 wen 为 0), 不会向通用寄存器堆写入任何数据。lo_write 为 0, 所以 LO 寄存器不受影响, 保持原值。WB_over 为 1, 写回阶段一个周期完成。

总结 源寄存器的值 (rs_value) 在 ID 级从寄存器堆读出, 通过 alu_operand1 传递到 EXE 级, 在 EXE 级被选入 exe_result, 经过 MEM 级传递为 mem_result, 最后在 WB 级被写入 HI 寄存器。这是一个从通用寄存器到特殊寄存器的数据流动。

MTHI 从取指到写回需要 5 个时钟周期。在第 5 个周期的时钟上升沿, HI 寄存器被更新。如果后续有 MFHI 指令要读 HI, 需要等到这个更新完成。

5.5 MTLO (向 LO 写入)

指令概述与功能 MTLO, "Move To LO", 功能是将一个通用寄存器的值写入 LO 寄存器。它与 MTHI 完全对称, 只是操作的目标是 LO 而不是 HI。指令格式为 000000_sssss_00000_00000_00000_010011, rs 指定源寄存器, funct 为 010011。

MTLO 用于显式设置 LO 寄存器的值, 常与 MTHI 配合使用, 在某些算法中初始化或设置 64 位的 HI:LO 寄存器对。

MTLO 的数据通路与 MTHI 对称: rs_value 从寄存器堆读出, 通过 alu_operand1 传递到 EXE 级, 在 EXE 级被选入 lo_result, 经过 MEM 级传递, 最后在 WB 级写入 LO 寄存器。不再赘述。

5.6 MFC0 (从 CP0 读取)

指令概述与功能 MFC0, "Move From Coprocessor 0", 功能是从协处理器 0 (CP0) 的某个寄存器读取值, 并将其写入一个通用寄存器。CP0 是 MIPS 架构中的系统控制协处理器, 包含用于异常处理、内存管理、状态控制等的特殊寄存器。

指令格式为 010000_00000_ttttt_ddddd_00000_000_sss, op 为 010000 表示协处理器指令, rs 为 00000 表示 MFC0, rt 为目标通用寄存器, rd 和 sel 字段共同指定 CP0 寄存器的地址。完整的 CP0 寄存器地址是 8 位: rd[4:0], sel[2:0]。

在本实验中, 实现了三个 CP0 寄存器:

STATUS (12.0): 状态寄存器, 目前只实现 EXL 位
CAUSE (13.0): 原因寄存器, 包含异常码
EPC (14.0): 异常程序计数器, 保存异常返回地址

IF 级（取指阶段） 取指过程标准化。PC 指向 MFC0 指令，地址送到 ROM，等待一个周期得到指令数据。IF_over 变为 1，指令和 PC 通过 IF_ID_bus 传递。

ID 级（译码阶段） 相关代码：

Listing 13: ID

```

1 // decode.v
2 assign inst_MFC0 = (op == 6'b010000) & (rs==5'd0)
3 & sa_zero & (funct[5:3] == 3'b000);
4
5 assign inst_wdest_rt = ... | inst_MFC0; // 写入rt
6 assign inst_no_rt = ... | inst_MFC0; // 不读rt
7
8 assign mfc0 = inst_MFC0; // 1'b1
9 assign cp0r_addr = {rd, cp0r_sel}; // {inst[15:11], inst[2:0]}
10 assign rf_wen = 1'b1;
11 assign rf_wdest = rt; // 目标是rt寄存器
12
13 assign ID_EXE_bus = {...,
14     cp0r_addr, // {rd, sel}
15     ...,
16     rf_wen, // 1'b1
17     rf_wdest, // rt
18     pc};

```

译码器识别 MFC0: op 为 010000（协处理器），rs 为 00000（MFC0 操作），sa 为 0，funct 的高 3 位为 000。inst_MFC0 信号置为 1。

MFC0 被分类为 inst_wdest_rt，因为目标寄存器是 rt 字段。rf_wen 为 1，rf_wdest 为 rt。

关键信号是 mfc0 置为 1，mtc0 为 0。cp0r_addr 被设置为 rd, cp0r_sel，即 inst[15:11], inst[2:0]，这是一个 8 位的 CP0 寄存器地址。

ID_EXE_bus 打包时，关键是 mfc0 为 1，cp0r_addr 为 rd, sel，rf_wen 为 1，rf_wdest 为 rt。当 ID_over 为 1 时，这些信号传递到执行阶段。由于 MFC0 不读源寄存器（除了固定的 CP0 寄存器），数据相关检测通常不会阻塞 MFC0。ID 阶段一个周期完成。

EXE 级（执行阶段） 执行阶段对 MFC0 透明。没有 ALU 运算，没有乘法。EXE_over 立即等于 EXE_valid。mfc0 信号和 cp0r_addr 继续在 EXE_MEM_bus 中传递，保持不变。执行阶段一个周期。

MEM 级（访存阶段） 访存阶段同样透明。不访问数据内存，dm_wen 为 0。MEM_over 立即等于 MEM_valid。mfc0 和 cp0r_addr 在 MEM_WB_bus 中继续传递。rf_wen 保持为 1，rf_wdest 保持为 rt。访存阶段一个周期。

WB 级（写回阶段） 相关代码：

Listing 14: WB

```

1 // wb.v

```

```

2 // CP0 寄存器读取
3 assign cp0r_rdata = (cp0r_addr=={5'd12,3'd0}) ? cp0r_status :
4                 (cp0r_addr=={5'd13,3'd0}) ? cp0r_cause :
5                 (cp0r_addr=={5'd14,3'd0}) ? cp0r_epc : 32'd0;
6
7 // 选择写回数据
8 assign rf_wdata = mfhi ? hi :
9                 mflo ? lo :
10                mfc0 ? cp0r_rdata : // 选择CP0寄存器的值
11                mem_result;
12
13 assign rf_wen = wen & WB_over; // 1'b1
14 assign rf_wdest = wdest; // rt
15 // 将CP0[rd,sel]的值写入GPR[rt]

```

WB 模块解包 MEM_WB_bus_r, 得到 mfc0 为 1, cp0r_addr 指定要读的 CP0 寄存器。

CP0 寄存器的读取是通过组合逻辑实现的, 这是一个多路选择器。根据 cp0r_addr 的值, 选择相应的 CP0 寄存器。如果地址是 12 (STATUS), 返回 cp0r_status 的值; 如果是 13 (CAUSE), 返回 cp0r_cause; 如果是 14 (EPC), 返回 cp0r_epc; 如果是其他未实现的地址, 返回 0。

cp0r_status、cp0r_cause 和 cp0r_epc 是从相应的寄存器构造的 32 位值。

rf_wdata 的选择逻辑中, mfhi 为 0, mflo 为 0, mfc0 为 1, 所以 rf_wdata 选择 cp0r_rdata。这个值就是从 CP0 读出的寄存器内容。

rf_wen 为 1, rf_wdest 为 rt, 所以在时钟上升沿, cp0r_rdata 的值被写入 GPR[rt]。整个过程不修改任何 CP0 寄存器, 因为 MFC0 只是读取。WB_over 为 1, 写回完成。

总结 在 WB 级, 根据 cp0r_addr 从 CP0 寄存器中选择一个值 (cp0r_rdata), 通过 rf_wdata 写入通用寄存器堆的 rt 号寄存器。中间各级只负责传递控制信号 mfc0 和地址 cp0r_addr。

5.7 MTC0 (向 CP0 写入)

指令概述与功能 MTC0, "Move To Coprocessor 0", 功能是将一个通用寄存器的值写入 CP0 的某个寄存器。它与 MFC0 相反: MFC0 从 CP0 读到通用寄存器, MTC0 从通用寄存器写到 CP0。

指令格式为 010000_00100_ttttt_ddddd_00000_000_sss, op 为 010000, rs 为 00100 表示 MTC0 操作, rt 为源通用寄存器, rd 和 sel 指定 CP0 寄存器地址。

在本实验中, STATUS 和 EPC 可以通过 MTC0 写入, 但 CAUSE 是只读的。

IF 级 (取指阶段) 取指过程标准化。PC 指向 MTC0 指令, 地址送到 ROM, 一个周期后得到指令。IF_over 变为 1, 指令和 PC 通过 IF_ID_bus 传递。

ID 级 (译码阶段) 相关代码:

Listing 15: ID

```

1 // decode.v
2 assign inst_MTC0 = (op == 6'b010000) & (rs==5'd4)
3                 & sa_zero & (funct[5:3] == 3'b000);
4

```

```

5      assign inst_no_rs = inst_MTC0 | ...; // rs域虽然是00100, 但不读寄存器
6
7      assign mtc0 = inst_MTC0; // 1'b1
8      assign cp0r_addr = {rd, cp0r_sel};
9      assign rf_wen = 1'b0; // 不写通用寄存器
10     assign rf_wdest = 5'd0;
11
12     // 需要读rt的值 (要写入CP0的数据)
13     assign alu_operand2 = rt_value;
14
15     assign ID_EXE_bus = {multiply, // 1'b0
16         ...,
17         alu_operand2, // rt_value
18         ...,
19         mtc0, // 1'b1
20         mfc0, // 1'b0
21         cp0r_addr,
22         ...,
23         rf_wen, // 1'b0
24         rf_wdest, // 5'd0
25         pc};

```

译码器识别 MTC0: op 为 010000, rs 为 00100 (这是固定值, 表示 MTC0 操作), sa 为 0, funct 高 3 位为 000。inst_MTC0 信号置为 1。

关键的区别是, MTC0 需要读取 rt 的值 (不是 rs)。rt 字段指定源寄存器, 包含要写入 CP0 的数据。译码器将 rt 送到寄存器堆的 raddr2 端口, rt_value 是读出的数据。

mtc0 信号置为 1, mfc0 为 0。cp0r_addr 设置为 rd, cp0r_sel。rf_wen 为 0 (不写通用寄存器), rf_wdest 为 0。

alu_operand2 被设置为 rt_value, 这是要写入 CP0 的数据。通过 ALU 操作数通路传递数据, 虽然 ALU 本身不使用。

ID_EXE_bus 打包时, 关键是 mtc0 为 1, alu_operand2 为 rt_value, cp0r_addr 为 rd, sel, rf_wen 为 0。

EXE 级 (执行阶段) 相关代码:

Listing 16: EXE

```

1      // exe.v
2      assign exe_result = mthi ? alu_operand1 :
3          mtc0 ? alu_operand2 : // rt_value
4          multiply ? product[63:32] : alu_result;
5
6      assign EXE_over = EXE_valid;
7
8      assign EXE_MEM_bus = {...,
9          exe_result, // rt_value (要写入CP0)
10         ...,
11         mtc0, // 1'b1

```

```

12     mfc0,          // 1'b0
13     cp0r_addr,
14     ...,
15     rf_wen,        // 1'b0
16     rf_wdest,      // 5'd0
17     pc};

```

对于 MTC0, mthi 为 0, mtc0 为 1, 所以 exe_result 等于 alu_operand2, 也就是 rt_value。这个值将被传递到后续阶段, 最终写入 CP0 寄存器。

EXE_over 立即等于 EXE_valid, 一个周期完成。EXE_MEM_bus 打包时, exe_result 包含 rt_value, mtc0 为 1, cp0r_addr 保持不变。这些信号传递到访存阶段。

MEM 级(访存阶段) 相关代码:访存阶段透明,不访问内存。mem_result 等于 exe_result(rt_value), 这个值继续传递到写回阶段。

mtc0 和 cp0r_addr 在 MEM_WB_bus 中继续传递。MEM_over 立即等于 MEM_valid, 一个周期完成。

WB 级(写回阶段) 相关代码:

Listing 17: WB

```

1  // wb.v
2  // CP0写使能
3  assign status_wen = mtc0 & (cp0r_addr=={5'd12,3'd0});
4  assign epc_wen = mtc0 & (cp0r_addr=={5'd14,3'd0});
5  // CAUSE寄存器是只读的, 没有cause_wen
6
7  // STATUS寄存器写入
8  always @(posedge clk) begin
9      if (!resetn || eret) begin
10         status_exl_r <= 1'b0;
11     end
12     else if (syscall) begin
13         status_exl_r <= 1'b1;
14     end
15     else if (status_wen) begin // MTC0写STATUS
16         status_exl_r <= mem_result[1]; // rt_value[1]
17     end
18 end
19
20 // EPC寄存器写入
21 always @(posedge clk) begin
22     if (syscall) begin
23         epc_r <= pc;
24     end
25     else if (epc_wen) begin // MTC0写EPC
26         epc_r <= mem_result; // rt_value
27     end

```



```

28     end
29
30     // 不写通用寄存器 (rf_wen=0)

```

写回阶段是 MTC0 真正修改 CP0 寄存器的地方。WB 模块解包 MEM_WB_bus_r，得到 mtc0 为 1，cp0r_addr 指定要写的 CP0 寄存器，mem_result 包含要写入的值 (rt_value)。

首先，生成各个 CP0 寄存器的写使能信号：如果 mtc0 为 1 且地址是 12 (STATUS)，status_wen 为 1；如果地址是 14 (EPC)，epc_wen 为 1。注意没有 cause_wen，因为 CAUSE 在本设计中是只读的。

STATUS 寄存器的更新逻辑是一个 always 块，有多个条件分支，优先级从高到低是：复位、ERET、SYSCALL、MTC0。如果 status_wen 为 1，status_exl_r 被更新为 mem_result 的第 1 位，也就是 rt_value[1]。程序可以通过 MTC0 显式地设置或清除 EXL 位。

EPC 寄存器的更新逻辑中，SYSCALL 的优先级高于 MTC0。如果 epc_wen 为 1 且没有 SYSCALL，epc_r 被更新为 mem_result，也就是完整的 rt_value。程序可以通过 MTC0 设置 EPC，这在某些异常处理场景中很有用。

MTC0 不写通用寄存器堆，rf_wen 为 0。WB_over 为 1，写回完成。

总结 rt 寄存器的值 (rt_value) 在 ID 级从寄存器堆读出，通过 alu_operand2 传递到 EXE 级，在 EXE 级被选入 exe_result，经过 MEM 级传递为 mem_result，最后在 WB 级根据 cp0r_addr 被写入相应的 CP0 寄存器。

5.8 SYSCALL (系统调用)

指令概述与功能 SYSCALL 是系统调用指令，用于从用户程序请求操作系统服务。它触发一个异常，将控制权从用户模式转移到内核模式，跳转到异常处理程序入口。这是用户程序与操作系统内核交互的主要机制。

指令格式为 000000_XXXXXXXXXXXXXXXXXXXX_001100，op 为 0，funct 为 001100。中间的 20 位通常包含一个系统调用码，但在硬件层面这些位被忽略，系统调用码通常通过寄存器（如 \$v0）传递。

SYSCALL 的执行过程涉及多个步骤：

1. 保存当前 PC 到 EPC 寄存器
2. 设置 CAUSE 寄存器的 ExcCode 为 8 (系统调用异常)
3. 设置 STATUS 寄存器的 EXL 位为 1 (进入异常级别)
4. 跳转到异常处理程序入口 (本实验中为地址 0x00000000)
5. 清空流水线中的其他指令 (cancel)

IF 级 (取指阶段) 取指过程标准化。PC 指向 SYSCALL 指令，地址送到 ROM，一个周期后得到指令。IF_over 变为 1，指令和 PC 通过 IF_ID_bus 传递到译码阶段。

ID 级 (译码阶段) 相关代码：

Listing 18: ID

```

1 // decode.v
2 assign inst_SYSCALL = (op == 6'b000000) & (funct == 6'b001100);
3
4 assign inst_no_rs = inst_MTC0 | inst_SYSCALL | inst_ERET;
5 assign inst_no_rt = ... | inst_SYSCALL;
6
7 assign syscall = inst_SYSCALL; // 1'b1
8 assign rf_wen = 1'b0;          // 不写寄存器
9 assign rf_wdest = 5'd0;
10
11 assign ID_EXE_bus = {...,
12     syscall,    // 1'b1
13     eret,       // 1'b0
14     rf_wen,     // 1'b0
15     rf_wdest,   // 5'd0
16     pc};        // 当前PC

```

译码器识别 SYSCALL: op 为 0, funct 为 001100。inst_SYSCALL 信号置为 1。识别条件很简单,只需要匹配这两个字段。

inst_no_rs 和 inst_no_rt 都为真,因为 SYSCALL 不需要从寄存器堆读取任何操作数。指令中的所有字段(除了 op 和 funct)都不重要,可以是任意值。

syscall 信号置为 1,这是关键的控制信号。

所有 ALU、乘法、内存相关的信号都不重要,因为 SYSCALL 不进行任何运算或访存操作。

ID_EXE_bus 打包时,关键是 syscall 为 1,还有 pc 的值(这个 PC 将最终被写入 EPC)。

EXE 级(执行阶段) 执行阶段对 SYSCALL 完全透明。没有 ALU 运算,没有乘法,没有特殊的计算。EXE_over 立即等于 EXE_valid,一个周期完成。

syscall 信号和 pc 值在 EXE_MEM_bus 中继续传递,保持不变。执行阶段本质上只是一个流水级,让 SYSCALL 指令继续向后推进。

MEM 级(访存阶段) 访存阶段同样透明。不访问内存,dm_wen 为 0。MEM_over 立即等于 MEM_valid,一个周期。syscall 和 pc 在 MEM_WB_bus 中继续传递。SYSCALL 指令继续向写回阶段推进。

WB 级(写回阶段) 写回阶段是 SYSCALL 真正起作用的地方,所有的异常处理逻辑都在这里实现。首先,更新 STATUS 寄存器的 EXL 位:

Listing 19: WB-exl

```

1 always @(posedge clk) begin
2     if (!resetn || eret) begin
3         status_exl_r <= 1'b0;
4     end
5     else if (syscall) begin
6         status_exl_r <= 1'b1; // 进入异常级别
7     end

```

```

8     else if (status_wen) begin
9         status_exl_r <= mem_result[1];
10    end
11 end

```

当 syscall 为 1 时，在时钟上升沿，status_exl_r 被设置为 1。这标志着处理器进入异常级别。

其次，更新 CAUSE 寄存器的 ExcCode：

Listing 20: WB-exccode

```

1    always @(posedge clk) begin
2        if (syscall) begin
3            cause_exc_code_r <= 5'd8; // 系统调用异常码
4        end
5    end

```

ExcCode 被设置为 8，这是 MIPS 架构中系统调用异常的标准编码。异常处理程序可以通过读取 CAUSE 寄存器来确定异常类型。

第三，保存异常返回地址到 EPC：

Listing 21: WB-epc

```

1    always @(posedge clk) begin
2        if (syscall) begin
3            epc_r <= pc; // 保存 SYSCALL 指令的 PC
4        end
5        else if (epc_wen) begin
6            epc_r <= mem_result;
7        end
8    end

```

pc 的值（SYSCALL 指令的地址）被保存到 epc_r。将来执行 ERET 指令时，会从 EPC 读取这个地址并跳转回去，实现异常返回。

第四，生成 exc_bus 信号：

Listing 22: WB-excbus

```

1    assign exc_valid = (syscall | eret) & WB_valid;
2    assign exc_pc = syscall ? EXC_ENTER_ADDR : cp0r_epc;
3    assign exc_bus = {exc_valid, exc_pc};

```

exc_valid 被置为 1，表示有异常发生。exc_pc 被设置为 EXC_ENTER_ADDR，在本实验中定义为 32'd0，即地址 0x00000000。exc_bus 这个 33 位信号（1 位 valid + 32 位地址）被送到 IF 模块，指示下一个 PC 应该跳转到异常入口。

第五，生成 cancel 信号：

Listing 23: WB-cancel

```

1    assign cancel = (syscall | eret) & WB_over;

```

cancel 信号被置为 1，表示需要取消流水线中的其他指令。这个信号会传播到所有流水级，清空它们的 valid 寄存器。

详细探究一下流水线清空机制：

cancel 信号的作用是清空流水线，确保 SYSCALL 之后的指令不会被执行。这是必要的，因为 PC 将跳转到异常处理程序，原本的程序流被打断。在 pipeline_cpu 模块中，各级的 valid 寄存器更新逻辑都包含 cancel 条件：

Listing 24: valid-cancel

```

1  always @(posedge clk) begin
2      if (!resetn || cancel) begin
3          ID_valid <= 1'b0;
4      end
5      else if (ID_allow_in) begin
6          ID_valid <= IF_over;
7      end
8  end

```

类似的逻辑应用于 EXE_valid、MEM_valid 和 WB_valid。当 cancel 为 1 时，在时钟上升沿，所有这些 valid 信号都被清零。这意味着流水线中的所有指令都被标记为无效，它们不会继续执行或产生任何效果。

SYSCALL 不写通用寄存器堆，rf_wen 为 0。WB_over 为 1，写回完成。

提问：如果发生嵌套异常，会发生什么？ 嵌套异常——在异常处理程序中再次发生异常。在本实验的简化设计中，STATUS 的 EXL 位已经是 1，再次发生 SYSCALL 会再次设置 EXL 为 1（无影响），但 EPC 会被覆盖为新的 PC 值，这就丢失了原来的返回地址。

5.9 ERET（异常返回）

指令概述与功能 ERET, "Exception Return", 用于从异常处理程序返回到被中断的程序。它执行以下操作：

1. 清除 STATUS 寄存器的 EXL 位（退出异常级别）
2. 从 EPC 寄存器读取返回地址
3. 跳转到 EPC 指定的地址
4. 清空流水线中的其他指令（cancel）

ERET 通常是异常处理程序的最后一条指令，标志着异常处理的完成。

指令格式为 010000_10000_00000_00000_00000_011000，op 为 010000（协处理器指令），rs 为 10000（ERET 操作的标志），其他字段都是 0，funct 为 011000。这是一个非常特殊的指令格式，容易识别。

IF 级（取指阶段） 取指过程标准化。PC 指向 ERET 指令（通常在异常处理程序的末尾），地址送到 ROM，一个周期后得到指令。IF_over 变为 1，指令和 PC 通过 IF_ID_bus 传递。

ERET 指令的 PC 值在这里不太重要，因为不需要保存，EPC 已经包含了要返回的地址。

ID 级（译码阶段） 译码器识别 ERET: op 为 010000, rs 为 10000 (16), rt 为 0, rd 为 0, sa 为 0, funct 为 011000。inst_ERET 信号置为 1。

inst_no_rs 和 inst_no_rt 都为真, ERET 不需要读任何寄存器。eret 信号置为 1, syscall 为 0。rf_wen 为 0 (不写通用寄存器), rf_wdest 为 0。所有运算和访存相关的信号都不重要。ID_EXE_bus 打包时, 关键是 eret 为 1, syscall 为 0。

EXE 级（执行阶段） 执行阶段对 ERET 完全透明。没有任何运算或操作。EXE_over 立即等于 EXE_valid, 一个周期。eret 信号在 EXE_MEM_bus 中继续传递, 保持为 1。ERET 指令向写回阶段推进。

MEM 级（访存阶段） 访存阶段同样透明。不访问内存。MEM_over 立即等于 MEM_valid, 一个周期。eret 在 MEM_WB_bus 中继续传递。ERET 指令到达写回阶段。

WB 级（写回阶段） 写回阶段是 ERET 的关键所在。

首先, 清除 STATUS 寄存器的 EXL 位:

Listing 25: WB-exl

```

1  always @(posedge clk) begin
2      if (!resetn || eret) begin
3          status_exl_r <= 1'b0; // 退出异常级别
4      end
5      else if (syscall) begin
6          status_exl_r <= 1'b1;
7      end
8      else if (status_wen) begin
9          status_exl_r <= mem_result[1];
10     end
11 end

```

复位和 eret 的优先级最高。当 eret 为 1 时, 在时钟上升沿, status_exl_r 被清零。这标志着处理器退出异常级别, 回到正常模式。

其次, 生成 exc_bus 信号:

Listing 26: WB-excbus

```

1  assign exc_valid = (syscall | eret) & WB_valid;
2  assign exc_pc = syscall ? EXC_ENTER_ADDR : cp0r_epc;
3  assign exc_bus = {exc_valid, exc_pc};

```

exc_valid 被置为 1, 表示需要进行 PC 跳转。exc_pc 被设置为 cp0r_epc, 即 EPC 寄存器保存的异常返回地址。exc_bus 这个 33 位信号被送到 IF 模块, 指示下一个 PC 应该跳转到 EPC 指定的地址。

第三, 生成 cancel 信号。

探究一下 PC 跳转机制: exc_bus 信号连接到 IF 模块, 在 IF 模块中:

Listing 27: WB-PCskip

```

1  assign {exc_valid, exc_pc} = exc_bus;

```

```
2
3  assign next_pc = exc_valid ? exc_pc :      // ERET: 跳转到EPC
4  jbr_taken ? jbr_target :
5  seq_pc;
```

由于 `exc_valid` 为 1, `next_pc` 被设置为 `exc_pc`, 即 `cp0r_epc` 的值。在时钟上升沿, PC 被更新为这个返回地址。

ERET 不写通用寄存器堆, `rf_wen` 为 0。不修改 HI/LO 寄存器。`WB_over` 为 1, 写回完成。

5.10 新增指令总结

以上九条新增指令构成了一个完整的乘法和异常处理系统:

乘法运算子系统 - MULT: 执行有符号乘法, 结果存入 HI:LO

- MFHI/MFLO: 从 HI/LO 读取乘法结果到通用寄存器
- MTHI/MTLO: 将通用寄存器的值写入 HI/LO

这五条指令使 CPU 具备了完整的乘法功能。乘法器采用迭代算法, HI 和 LO 寄存器提供了存储 64 位结果的能力, MFHI/MFLO/MTHI/MTLO 提供了访问这些寄存器的手段。

异常处理子系统 - MFC0/MTC0: 在通用寄存器和 CP0 寄存器之间传输数据

- SYSCALL: 触发系统调用异常
- ERET: 从异常返回

这四条指令使 CPU 具备了基本的异常处理能力。CP0 寄存器 (STATUS、CAUSE、EPC) 提供了异常状态的存储, SYSCALL 提供了触发异常的机制, ERET 提供了返回机制, MFC0/MTC0 提供了软件访问 CP0 寄存器的手段。这为实现操作系统的系统调用功能提供了基础。

6 总结感想

通过本次五级流水线 CPU 的复习分析实验, 学习到了很多上学期没能理解的知识点。

再一次修改源代码 bug, 这一次不再是一知半解的尝试, 而是意识到了 bug 的本质: 指令 ROM 的读取延迟与 `IF_over` 信号生成时机的不匹配。IP 核配置修改的方案和修改 `IF_over` 逻辑的方案都是在让 `IF_over` 信号与指令数据真正可用的时刻同步。后一种方案是从 github 上寻到的思路, 这也启发了我对问题本质的思考, 十分感谢。

五级流水线在原有已实现指令的基础上, 成功实现了乘法运算子系统 (MULT、MFHI、MFLO、MTHI、MTLO) 和异常处理子系统 (SYSCALL、ERET、MFC0、MTC0) 共 9 条关键指令, 使 CPU 从单纯的算术逻辑处理器演进为具备操作系统支持能力的完整处理器。通过对每条指令的逐级分析, 巩固了我对五级流水线的认知和指令集设计的知识。

在分析中也发现了些当前实现存在的问题, 比如 HI 读写的相关性问题和嵌套异常问题, 这是当前设计比较简陋导致的, 很多问题以及处理效率问题都需要未来的设计来优化。